

Werkplaats 6 Herkansing



Datum: 12-04-2026

Studentnummer: 0894873

Naam: Stephan Kuil

Vakcode: SWDWER61X2

Herkansingsopdracht: Real Time systeem WP6

Inhoudsopgave

Inleiding	2
Architectuurschema:.....	3
MQTT – Voor- en nadelen	4
Voordelen.....	4
Nadelen	5
Pygame – Voor- en nadelen	5
Voordelen.....	5
Nadelen	6
Pytest – Voor- en nadelen.....	7
Voordelen.....	7
Nadelen	8
PC Specificaties die ik gebruik voor de pacman game.	8
GPU	8
Besturingssysteem.....	8
Processor (CPU)	9
Geheugen (RAM)	9
Opslag / Virtueel Geheugen	9
Moederbord / Laptop.....	10
BIOS	10
Netwerk	10
Virtualisatie	11
Installatie	11
Testplan	11
1. move – Pacman beweegt correct	13
Unhappy paths (fouten/ uitzonderingen)	14
1. move – Pacman tegen muur	14
UI-Mockup.....	14
Dataformaten – JSON-structuren voor berichten.....	15
1. Player Input (beweging)	16
2. Game State (centrale bericht).....	16

3. Dot gegeten.....	17
4. Power-up gepakt	17
5. Ghost update (optioneel apart)	17
6. Player geraakt door ghost	18
7. Game start	18
8. Game over	18
Architectuur (belangrijk)	18

Link naar github repository

https://github.com/Stephankuil/WP6-Stephan_Kuil_0894873.git

Ga naar branche beoordeling.

De laatste commit van branche beoordeling met de tag v1.0 is de juiste versie.

Gegevens voor de .env

```
MQTT_BROKER_HOST=4abaa784a435421ba0190c9884dd2b89.s1.eu.hivemq.cloud
MQTT_BROKER_PORT=8883
MQTT_USERNAME=Stephan
MQTT_PASSWORD=Geheimwachtwoord1
```

Inleiding

Voor dit project wil ik graag de docenten bedanken voor hun inzet en begeleiding gedurende het proces. Met dit document en de bijbehorende opdracht hoop ik aan te tonen dat ik beschik over de kennis en vaardigheden die nodig zijn om het diploma Software Development te behalen.

In de afgelopen twee jaar heb ik veel geleerd op het gebied van programmeren en softwareontwikkeling. Ik ben gemotiveerd om hier mijn beroep van te maken en mij hier verder in te blijven ontwikkelen. Aan dit project heb ik veel tijd en energie besteed, en ik heb mij ingezet om een zo goed mogelijk resultaat neer te zetten.

Door omstandigheden binnen de samenwerking met mijn teamgenoot heb ik effectief slechts vier van de acht weken volledig aan het project kunnen werken. Hierdoor is het project niet volledig afgerond zoals oorspronkelijk gepland. Desondanks heb ik in de beschikbare tijd zo veel mogelijk functionaliteit gerealiseerd en mijn kennis toegepast.

Een van de beoordelende docenten gaf aan dat het toelichten van de werking van MQTT binnen mijn Pacman-project kan bijdragen aan het aantonen van mijn technische inzicht. Met deze toelichting wil ik laten zien dat ik de onderliggende concepten beheers en dat ik, met voldoende tijd, in staat ben geweest het project succesvol af te ronden.

Architectuurschema:

MQTT – Voor- en nadelen

Voordelen

1. Lichtgewicht protocol

MQTT gebruikt weinig bandbreedte en weinig data. Hierdoor werkt het goed op langzame of instabiele netwerken.

2. Publish/Subscribe model

Clients hoeven elkaar niet direct te kennen. Ze communiceren via een broker, wat het systeem schaalbaar maakt.

3. Real-time communicatie

Geschikt voor toepassingen waar berichten direct moeten worden doorgegeven, zoals games, IoT of sensoren.

4. Betrouwbaarheid via QoS levels

MQTT ondersteunt verschillende Quality of Service niveaus:

- QoS 0 – Fire and forget
- QoS 1 – At least once
- QoS 2 – Exactly once

5. Goed voor IoT en multiplayer

Veel gebruikt bij:

- smart homes
- sensornetwerken
- realtime games

Nadelen

1. Afhankelijk van een broker

Alle communicatie loopt via een MQTT-broker (bijv. HiveMQ of Mosquitto). Als de broker uitvalt, werkt het systeem niet.

2. Minder geschikt voor grote data

MQTT is bedoeld voor kleine berichten, niet voor grote bestanden.

3. Complexer debugging

Omdat communicatie via topics loopt, kan het soms lastiger zijn om fouten te traceren.

4. Security moet extra worden ingesteld

Encryptie en authenticatie moeten vaak handmatig worden geconfigureerd.

Pygame – Voor- en nadelen

Voordelen

1. Simpel om mee te beginnen

Pygame maakt het eenvoudig om games te bouwen met Python.

2. Veel tutorials en documentatie

Er is een grote community en veel voorbeelden.

3. Direct grafische output

Je kunt snel een game loop maken met:

- sprites
- geluid
- toetsenbord input

4. Goed voor onderwijs en prototypes

Veel gebruikt bij:

- programmeeronderwijs
- game prototypes
- kleine indie games

Nadelen

1. Minder performant dan game engines

Pygame draait op Python, wat langzamer is dan engines zoals Unity of Unreal.

2. Veel dingen moet je zelf bouwen

Er is geen ingebouwde:

- physics engine
- scene editor
- UI systeem

3. Minder geschikt voor grote games

Voor complexe 3D games is Pygame niet geschikt.

4. Geen moderne rendering engine

Grafische mogelijkheden zijn beperkter dan moderne game engines.

Pytest – Voor- en nadelen

Voordelen

1. Automatisch testen van code

Met Pytest kun je automatisch controleren of je code correct werkt.

2. Test-Driven Development (TDD)

Je kunt eerst tests schrijven en daarna de code implementeren.

3. Zeer leesbare tests

Tests zijn eenvoudig te schrijven:

```
def test_add():  
    assert add(2,3) == 5
```

4. Veel uitbreidingen (plugins)

Pytest heeft veel plugins zoals:

- coverage
- fixtures
- mocking
- onderhoudbaar

5. Snelle feedback bij fouten

Je ziet direct welke test faalt en waarom.

Nadelen

1. Extra ontwikkeltijd

Tests schrijven kost tijd, vooral bij kleine projecten.

2. Leercurve bij complexe tests

Fixtures, mocks en parametrized tests kunnen ingewikkeld worden.

3. Niet altijd nodig voor kleine scripts

Voor simpele scripts kan het overkill zijn.

4. Tests moeten onderhouden worden

Als de code verandert, moeten tests vaak worden aangepast.

PC Specificaties die ik gebruik voor de pacman game.

GPU

Intel(R) UHD Graphics 620

Besturingssysteem

- **OS:** Windows 11 Pro
- **Versie:** 10.0.22631 (Build 22631)
- **Architectuur:** 64-bit

Processor (CPU)

- **Model:** Intel Core (Intel64 Family 6 Model 142)
- **Kloksnelheid:** ~2.0 GHz
- **Aantal CPU's:** 1 processor

Geheugen (RAM)

- **Totaal:** 16 GB RAM
- **Beschikbaar (op moment van check):** ~6.4 GB

Opslag / Virtueel Geheugen

- **Pagefile:** C:\pagefile.sys

- **Virtueel geheugen totaal:** ~19 GB
- **Virtueel geheugen in gebruik:** ~10.6 GB

Moederbord / Laptop

- **Fabrikant:** Lenovo
- **Model:** 20KS001QMH

BIOS

- **Versie:** R0PET67W (1.44)
- **Datum:** 19 juni 2020

Netwerk

- **Wi-Fi:** Intel Dual Band Wireless-AC 3165
- **Ethernet:** Realtek PCIe GbE Family Controller
- **Bluetooth:** PAN Adapter

- **Virtuele netwerkadapter:** VirtualBox Host-Only Adapter

Virtualisatie

- **Hypervisor:** Actief (Hyper-V / virtualisatie ingeschakeld)

Installatie

- **Windows geïnstalleerd op:** 24 november 2022
- **Laatste reboot:** 7 maart 2026

Testplan

Welke testen moeten geschreven worden?

Voor iedere functie moet een Happy en Unhappy test geschreven worden.

Voorbeeld van methodes die getest kunnen worden:

`startGame()`

`updateGame()`

`movePacman()`

`moveGhosts()`

`checkCollision()`

`eatCheese()`

`updateScore()`

`spawnGhost()`

`spawnCheese()`

`resetGame()`

`gameOver()`

`detectGhostCollision()`

`updatePositions()`

`renderGame()`

Dit zijn 15 mogelijke methodes die dus een happy of Unhappy path hebben.

Dus dan zou je 30 tests voor kunnen schrijven.

Voorbeeld happy path test

Happy paths (verwachte/ correcte situaties)

1. move – Pacman beweegt correct

Test: Pacman beweegt naar rechts.

Given

Pacman staat op (5,5)

When

`move("right")`

Then

Pacman staat op (6,5)

Voorbeeld Unhappy path

Unhappy paths (fouten/ uitzonderingen)

1. move – Pacman tegen muur

Test: Pacman probeert door een muur te lopen.

Given

Pacman (2,2)

Muur (3,2)

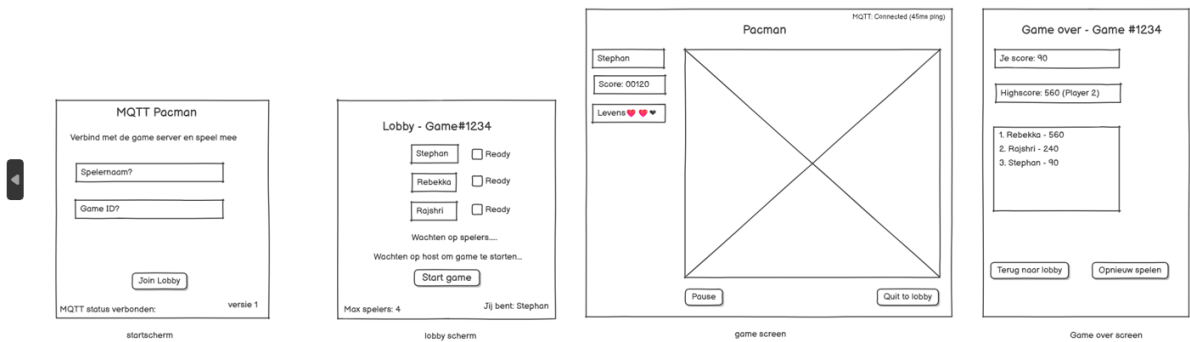
When

`move("right")`

Then

- positie verandert niet

UI-Mockup



Dataformaten – JSON-structuren voor berichten

1. Player Input (beweging)

Topic: pacman/{room}/input

```
{
  "player_id": "stephan",
  "direction": "LEFT",
  "timestamp": 1712938471
}
```

Velden:

- player_id: unieke spelernaam of ID
- direction: UP / DOWN / LEFT / RIGHT
- timestamp: handig voor synchronisatie

2. Game State (centrale bericht)

Topic: pacman/{room}/state

```
{
  "players": {
    "stephan": { "x": 5, "y": 10, "score": 120, "lives": 2 },
    "jan": { "x": 8, "y": 3, "score": 90, "lives": 3 }
  },
  "ghosts": [
    { "x": 7, "y": 7 },
    { "x": 10, "y": 4 }
  ],
  "dots": [
    [1, 1],
    [2, 5],
    [6, 8]
  ],
  "powerups": [
    { "x": 3, "y": 6, "active": true }
  ],
}
```

```
"power_mode": {  
  "active": true,  
  "player_id": "stephan",  
  "ends_at": 1712938500  
}  
}
```

Dit bericht bepaalt wat alle clients tekenen. De host stuurt dit periodiek.

3. Dot gegeten

Topic: pacman/{room}/dot_eaten

```
{  
  "player_id": "stephan",  
  "x": 5,  
  "y": 10,  
  "score": 130  
}
```

4. Power-up gepakt

Topic: pacman/{room}/powerup

```
{  
  "player_id": "stephan",  
  "x": 3,  
  "y": 6,  
  "duration_ms": 8000  
}
```

5. Ghost update (optioneel apart)

Topic: pacman/{room}/ghosts

```
{  
  "ghosts": [  
    { "id": 1, "x": 7, "y": 7, "direction": "LEFT" },  
  ]  
}
```

```
    { "id": 2, "x": 10, "y": 4, "direction": "UP" }  
  ]  
}
```

6. Player geraakt door ghost

Topic: pacman/{room}/hit

```
{  
  "player_id": "stephan",  
  "lives_left": 1,  
  "respawn": {  
    "x": 1,  
    "y": 1  
  }  
}
```

7. Game start

Topic: pacman/{room}/start

```
{  
  "started_by": "stephan",  
  "players": ["stephan", "jan"],  
  "start_time": 1712938471  
}
```

8. Game over

Topic: pacman/{room}/game_over

```
{  
  "winner": "stephan",  
  "scores": {  
    "stephan": 250,  
    "jan": 180  
  }  
}
```

Architectuur (belangrijk)

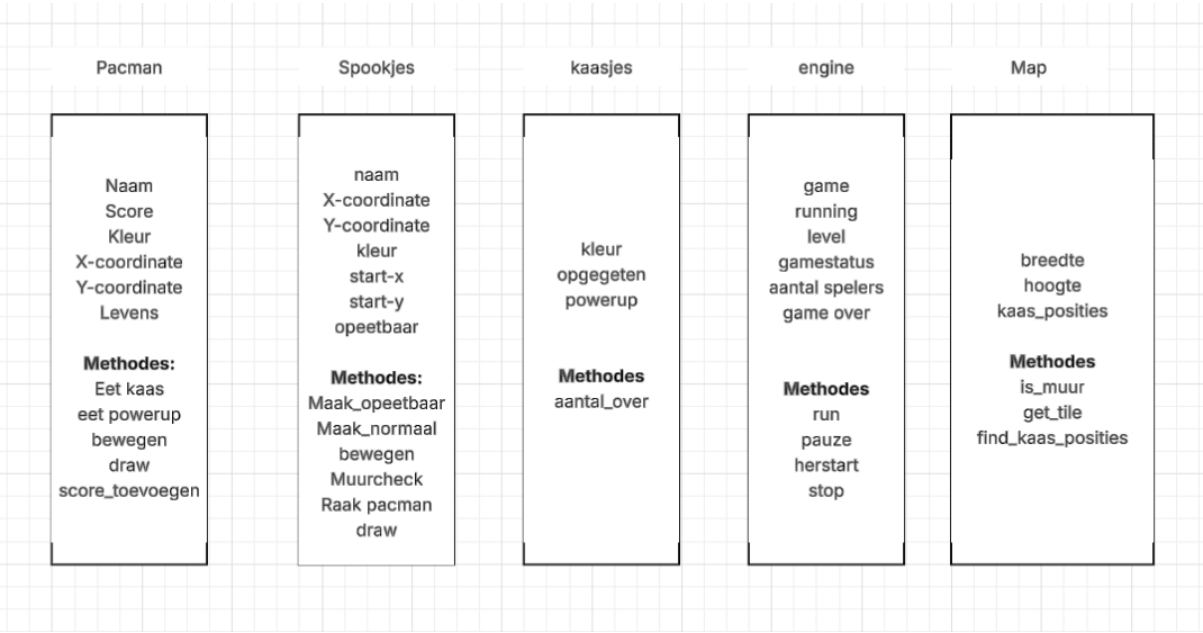
Host (server / game host):

- ontvangt input
- verwerkt game logica
- stuurt state

Clients:

- sturen alleen input
- renderen op basis van state

Klassendiagram



Gebruik van MQTT in de multiplayer Pacman game

Voor dit project was het de bedoeling om de Pacman game uit te breiden naar een multiplayer variant waarbij meerdere spelers tegelijkertijd in dezelfde gamewereld kunnen spelen. Om de communicatie tussen verschillende spelers en systemen mogelijk te maken, heb ik gekozen voor het gebruik van het **publish/subscribe-model via MQTT**.

Werking van MQTT binnen de game

MQTT is een lichtgewicht protocol dat veel gebruikt wordt voor real-time communicatie. In mijn applicatie zou een centrale MQTT-broker (bijvoorbeeld HiveMQ) fungeren als tussenstation waar alle clients (spelers) berichten naartoe sturen en van ontvangen.

Elke speler draait lokaal de game en maakt verbinding met de broker. Binnen de game wordt gewerkt met zogenaamde *topics*, bijvoorbeeld:

- pacman/{room}/input
- pacman/{room}/state
- pacman/{room}/events

Hierbij staat {room} voor een unieke game-sessie, zodat meerdere games tegelijkertijd kunnen bestaan.

Architectuur van de multiplayer game

De multiplayer functionaliteit is gebaseerd op een **client-host model**:

- **Host (gameleider):**
 - Ontvangt input van alle spelers via MQTT
 - Verwerkt de volledige game logica (beweging, botsingen, score, power-ups)
 - Stuurt regelmatig de volledige game state terug naar alle spelers
- **Clients (spelers):**
 - Sturen alleen hun input (bijvoorbeeld richting van Pacman)
 - Ontvangen de game state van de host
 - Renderen de game op basis van deze state

Deze aanpak zorgt ervoor dat er één centrale waarheid is (de host), wat synchronisatieproblemen voorkomt.

Voorbeeld van communicatie

1. Een speler drukt op een toets (bijvoorbeeld naar links bewegen)
2. De client stuurt een MQTT-bericht naar het topic `input`
3. De host ontvangt dit bericht en verwerkt de beweging
4. De host berekent de nieuwe game state
5. De host publiceert de nieuwe state naar alle clients
6. Alle clients updaten hun scherm op basis van deze state

Waarom MQTT

De keuze voor MQTT is gemaakt vanwege de volgende voordelen:

- Lage latency, geschikt voor real-time games
- Eenvoudig publish/subscribe model
- Schaalbaar naar meerdere spelers
- Losgekoppelde communicatie tussen clients

Doel van de implementatie

Het doel van deze aanpak was om een werkende multiplayer Pacman game te realiseren waarbij meerdere spelers via het netwerk kunnen deelnemen aan één gedeelde game. Door gebruik te maken van MQTT wilde ik laten zien dat ik in staat ben om:

- real-time communicatie te implementeren
- een schaalbare architectuur op te zetten
- netwerkklogica te scheiden van game logica

Hoewel de volledige implementatie door tijdsgebrek niet volledig is afgerond, laat dit ontwerp zien hoe de multiplayer functionaliteit technisch gerealiseerd had kunnen worden.

Reflectie

Wat heb ik geleerd over MQTT-communicatie?

Tijdens dit project heb ik geleerd dat het belangrijk is om al vanaf het begin rekening te houden met MQTT-communicatie. Het is niet iets dat je eenvoudig achteraf kunt toevoegen. De structuur van je applicatie moet hier vanaf het begin op ingericht zijn, omdat de manier waarop data wordt verstuurd en verwerkt invloed heeft op de hele architectuur van de game.

Wat was het moeilijkst?

Het moeilijkste onderdeel vond ik de voorbereidingsfase. In deze fase wist ik nog niet precies hoe ik het project technisch moest opzetten. Ik merk dat ik het lastig vind om vooraf alles uit te denken, omdat ik juist tijdens het programmeren op nieuwe ideeën kom. Hierdoor veranderde mijn aanpak regelmatig, wat het moeilijker maakte om een duidelijke structuur aan te houden.

Hoe zou je het project verbeteren als je meer tijd had?

Als ik meer tijd had gehad, zou ik het project verder uitbreiden met extra functionaliteiten. Zo zou ik:

- Geluidseffecten toevoegen om de game levendiger te maken
- Een live scoreboard implementeren zodat spelers real-time scores kunnen zien
- Meerdere levels toevoegen, zoals level 2 en level 3, om de game uitdagender te maken

